GRAPH NEURAL NETWORKS FOR SYSTEM CALL GRAPH-BASED INTRUSION DETECTION

Raphaël Faure & Tristan Beruard & Sacha Dupuydauby
CentraleSupélec
{raphael.faure2, tristan.beruard, sacha.dupuydauby}@student-cs.fr

ABSTRACT

Host-based intrusion detection aims to detect malicious executions from low-level traces of process activity. System calls are a natural signal for this objective because they directly expose how a program interacts with the operating system. In this report, we study how the choice of representation changes the learning problem and how this choice affects robustness across tasks. We compare graph-based and sequence-based models on ADFA-LD Creech & Hu (2013) and two LID-DS scenarios et al. (2021). Our graph pipeline converts each trace into a directed weighted syscall graph and evaluates a classical PageRank-based anomaly detector, several graph neural networks (GCN Kipf & Welling (2017), GraphSAGE Hamilton et al. (2017), GAT Veličković et al. (2018), WGCN+), and a Temporal Graph Network (TGN) **?** that processes each trace as an ordered stream of timestamped events rather than an aggregated static graph. We also benchmark sequence-only baselines to situate the graph approach with respect to the historical syscall-sequence literature Hofmeyr et al. (1998). The experiments show that graph representations are informative, but their advantage depends on the regime: GAT is strongest on ADFA-LD, whereas the PageRank and TF-IDF baselines are extremely competitive on the bruteforce LID-DS scenario. TGN, which uniquely exploits per-event timestamps available in LID-DS, provides a complementary temporal view but is outperformed by static graph methods, confirming that these attacks manifest as frequency changes rather than temporal ordering changes. A transfer experiment across LID-DS attack families further shows that all methods degrade substantially out of distribution, with PageRank and WGCN+ remaining the most stable.

1 INTRODUCTION/MOTIVATION

Host-based intrusion detection remains difficult because malicious activity often appears as a subtle deformation of otherwise legitimate execution traces. System call logs are attractive in this setting because they are available at the operating-system boundary and provide a detailed view of process behavior. Early work therefore modeled them directly as short sequences or sequence windows Hofmeyr et al. (1998); Warrender et al. (1999). More recent graph-based work argues that the transition structure itself is discriminative and should be modeled explicitly Grimmer et al. (2018). However, it remains unclear when a graph representation is actually more useful than a direct sequential one for host-based intrusion detection.

Our starting point is that a syscall trace is not only a sequence, but also a transition system. If two syscalls interact repeatedly, if a subset of calls becomes a hub, or if unusual loops emerge, this structure is naturally visible in a graph representation. This motivates our main research questions: does a syscall-graph representation improve host-based intrusion detection, which graph models are the most relevant for these data, and how stable are these conclusions across datasets and attack scenarios?

The practical motivation is straightforward. Better host-based intrusion detection can support endpoint protection, anomaly triage, incident response, and post-compromise forensics. From a machine learning perspective, the project also provides a useful case study for comparing interpretable graph scoring methods with learned graph representations on a real cybersecurity task.

2 PROBLEM DEFINITION

Let a syscall trace be a sequence $s = (x_1, \ldots, x_T)$, where each token x_t is either an integer syscall identifier, as in ADFA-LD, or a syscall name extracted from a raw trace, as in LID-DS. From each sequence we build a directed weighted graph $G = (V, E, w)$ where each node in V is a unique syscall and each edge $(u, v) \in E$ records the number of times syscall u is immediately followed by syscall v in the trace.

The supervised task is binary graph classification. Given a trace graph G_i and its label $y_i \in \{0, 1\}$, with 0 denoting normal behavior and 1 an attack trace, we seek a predictor $f(G_i) \to \hat{y}_i$. Because the datasets are imbalanced and scenario-dependent, we evaluate the predictor with accuracy, precision, recall, F1 score, and ROC-AUC rather than relying on accuracy alone.

We also consider an anomaly-detection baseline based on PageRank. In that setting, the training procedure builds a normal library from benign traces only and the decision is taken from an anomaly score computed on sliding windows. Concretely, for a fixed window length m, a trace $s = (x_1, \ldots, x_T)$ is decomposed into overlapping subsequences $w_t = (x_t, \ldots, x_{t+m-1})$ for $t = 1, \ldots, T - m + 1$. Each window is compared with a library of normal patterns, and the final anomaly score is the fraction of windows whose distance to that library exceeds a chosen threshold. The problem then becomes threshold-based anomaly scoring instead of direct discriminative classification.

3 RELATED WORK

System call based intrusion detection has historically been studied through sequence models and symbolic matching rules because the data naturally arrives as an ordered stream. The classical UNM line of work Hofmeyr et al. (1998); Warrender et al. (1999) showed that short syscall windows already provide a strong anomaly-detection signal, which remains an important baseline for any newer approach. Graph-based approaches instead argue that transition structure itself is discriminative and should therefore be modeled explicitly. The system-call graph formulation of Grimmer et al. Grimmer et al. (2018) is particularly relevant here because it motivates representing a trace by its directed transition graph rather than by sequence statistics only.

On the representation-learning side, graph neural networks provide a natural way to combine local transition structure with node-level attributes. GCN Kipf & Welling (2017) is a standard baseline based on neighborhood aggregation, while GraphSAGE Hamilton et al. (2017) and GAT Veličković et al. (2018) extend this idea respectively through flexible aggregation and attention over neighbors. Although these families are widely used in graph learning, they remain comparatively underexplored for host-based intrusion detection on syscall graphs.

Our classical baseline is related to the PageRank anomaly scoring strategy of Qian et al. (2013). The original idea is to derive transition suspiciousness from a PageRank-weighted normal graph and then measure the distance between observed windows and a library of normal patterns. Our implementation keeps this core mechanism but adapts it to a multi-trace training pipeline and to reproducible Hydra-based experiments.

4 METHODOLOGY

Datasets and preprocessing. We use two host-based intrusion detection datasets. ADFA-LD Creech & Hu (2013) provides integer-encoded syscall traces with separate normal and attack folders. LID-DS et al. (2021) provides richer per-sample archives containing raw `.sc` traces and metadata. For LID-DS, our final study focuses on the scenarios `Bruteforce_CWE-307` and `CVE-2014-0160`. Within each retained scenario, we use a standard stratified train/validation/test split.

Graph construction and node features. Each trace is converted into a directed weighted graph. For ADFA-LD, node features include relative frequency, in/out degree, weighted degree, PageRank, and self-loop indicators. For LID-DS, we enrich the representation with process diversity, return-value statistics, and temporal gap statistics recovered from the raw traces. More precisely,

the additional LID-DS features summarize how many distinct processes emit a syscall, how often it returns errors, the empirical moments of its return values, and the typical time gaps before and after this syscall. Each graph is then turned into a PyTorch Geometric object with both numeric node features and a learned embedding for the syscall identity. Unless otherwise stated, the default GNN configuration uses the full node-feature profile together with weighted edges.

Feature variation. The graph representation itself is a design choice, so we explicitly study how the selected node and edge features affect performance. On the node side, we consider progressively richer profiles. The simplest setting uses only the learned syscall identity embedding, which tests whether structural propagation alone is sufficient. A second family of profiles uses structural descriptors such as degree, weighted degree, PageRank, and self-loop indicators. On LID-DS, a richer profile additionally includes behavioral statistics extracted from the raw traces: error rates indicate whether a syscall often fails, return-value moments summarize the dispersion of its outputs, process diversity measures how many distinct processes or execution contexts emit it, and temporal gaps capture whether it tends to appear in bursts or after long pauses. On the edge side, we compare two regimes: binary edges, where an observed transition is represented only by its existence, and weighted edges, where the multiplicity of a transition is preserved. This ablation is important because syscall graphs can encode both *which* transitions exist and *how often* they occur, and these two signals need not contribute equally across scenarios.

PageRank baseline. The PageRank detector follows a classical anomaly-detection logic. We build a normal transition graph, compute PageRank scores, derive a suspiciousness map $k(u, v)$ from the weighted outgoing contribution of each transition, slide a fixed-size window over the observed sequence, and score each observed window by its minimum weighted distance to a normal pattern. The final anomaly score is the proportion of windows whose distance exceeds a threshold.

The production implementation makes three engineering adaptations. First, the pattern library is built from all normal training traces instead of a single file. Second, we use pattern subsampling, caching, and prefix-based candidate filtering to keep evaluation tractable on larger datasets. Third, the distance and anomaly thresholds are selected on the validation split rather than fixed manually. The mathematical core of the method is therefore preserved while the training protocol is generalized to our datasets.

Sequence-only baselines. To complement the graph-based view, we also evaluate models that operate directly on the syscall sequence without explicitly building a graph. These baselines are important because they connect our study to the historical syscall-sequence literature initiated by the UNM work Hofmeyr et al. (1998); Warrender et al. (1999). The first baseline is a TF-IDF representation of syscall n-grams followed by logistic regression. If $\phi(s_i)$ denotes the sparse TF-IDF vector extracted from a trace s_i, the classifier predicts

$$p(y_i = 1 \mid s_i) = \sigma(w^\top \phi(s_i) + b).$$

This baseline captures local order statistics and repeated motifs very efficiently, but it does not explicitly model long-range transition structure beyond the selected n-gram window.

The second baseline is a bidirectional GRU sequence encoder. Given embedded tokens $e(x_t)$, the recurrent state is updated as

$$h_t = \mathrm{GRU}(e(x_t), h_{t-1}),$$

and the final representation is passed to a small MLP classifier. This approach preserves sequential order more faithfully than TF-IDF, but in practice it requires truncating long traces to keep training tractable. In our implementation, this makes the GRU more sensitive to sequence length and to where discriminative events occur inside the trace.

GNN models. We compare four graph neural architectures. In all cases, each node v starts from an initial representation

$$h_v^{(0)} = [\, e(\mathrm{syscall}_v) \,\|\, x_v \,],$$

where $e(\mathrm{syscall}_v)$ is a learned embedding of the syscall identity and x_v is the handcrafted numeric feature vector. Let A denote the weighted adjacency matrix, with A_{uv} equal to the number of observed transitions from syscall u to syscall v, and let $\mathcal{N}(v)$ be the neighborhood of v.

The first baseline is the Graph Convolutional Network (GCN) Kipf & Welling (2017). In matrix form, a layer can be written as

$$H^{(\ell+1)} = \sigma\Big(\hat{D}^{-1/2}\hat{A}\hat{D}^{-1/2}H^{(\ell)}W^{(\ell)}\Big),$$

where $\hat{A} = A + I$ adds self-loops, $\hat{D}$ is the associated degree matrix, $W^{(\ell)}$ is a learnable weight matrix, and σ is a pointwise non-linearity. Intuitively, GCN smooths node representations over the local transition structure. This is relevant for syscall graphs because normal executions often induce repeated and locally coherent transition motifs. Its limitation is that all neighbors are mixed through the same normalized operator, which can blur rare but important malicious transitions.

The second baseline is GraphSAGE Hamilton et al. (2017), which separates the representation of a node from the aggregate of its neighbors:

$$m_v^{(\ell)} = \mathrm{MEAN}\{h_u^{(\ell)} : u \in \mathcal{N}(v)\}, \qquad h_v^{(\ell+1)} = \sigma\Big(W_{\mathrm{self}}^{(\ell)}h_v^{(\ell)} + W_{\mathrm{neigh}}^{(\ell)}m_v^{(\ell)}\Big).$$

This formulation is useful when one expects the node's own state and its neighborhood context to play different roles. Compared with GCN, GraphSAGE can also be slightly more robust to over-smoothing, that is, to the regime where repeated neighborhood averaging makes the embeddings of different nodes nearly indistinguishable after several layers. For syscall graphs, this matters because rare syscall roles can disappear if every node becomes too similar to its neighbors.

The third baseline is the Graph Attention Network (GAT) Veličković et al. (2018). Instead of averaging neighbors uniformly after normalization, GAT learns attention coefficients:

$$\alpha_{vu}^{(\ell)} = \mathrm{softmax}_{u\in\mathcal{N}(v)}\left(\mathrm{LeakyReLU}\Big(a^{\top}[Wh_v^{(\ell)} \| Wh_u^{(\ell)} \| \phi(e_{vu})]\Big)\right),$$

$$h_v^{(\ell+1)} = \sigma\left(\sum_{u\in\mathcal{N}(v)} \alpha_{vu}^{(\ell)} W h_u^{(\ell)}\right),$$

where $\phi(e_{vu})$ denotes the edge attribute derived from the transition weight. This mechanism is particularly appealing for intrusion detection because not all syscall transitions are equally informative. A few rare edges can carry a large fraction of the malicious signal, and attention allows the model to focus more strongly on these transitions. This is one reason why GAT is a plausible candidate on ADFA-LD, where attack traces often differ through specific localized changes in transition importance rather than through a uniform deformation of the whole graph.

Finally, our main architecture is denoted **WGCN+**. It keeps the weighted GCN propagation rule above, but adds two modifications tailored to the current data regime. First, after each message-passing layer we apply layer normalization, which stabilizes hidden representations when graph sizes and degrees vary across samples. Second, instead of a single global pooling operator, we concatenate mean pooling and max pooling:

$$g_G = \Big[\mathrm{MeanPool}\Big(H^{(L)}\Big) \,\|\, \mathrm{MaxPool}\Big(H^{(L)}\Big)\Big].$$

Mean pooling summarizes the average execution profile of the trace, while max pooling preserves the strongest activated motifs. This combination is well suited to syscall graphs because an attack can manifest both as a global redistribution of transition statistics and as a small set of highly abnormal local patterns. In practice, we expect GCN-like models to be favored when the signal is distributed across many transitions, and GAT to be favored when a small number of edges carry disproportionate discriminative information.

Temporal Graph Network. Static syscall graphs collapse all transition events into a single weighted adjacency matrix, discarding the order and timing of individual events. To address this, we additionally evaluate a Temporal Graph Network (TGN) **?** applied directly to LID-DS, which is the only dataset in our benchmark that provides per-event nanosecond timestamps.

Each execution trace is represented as an ordered sequence of temporal edges (u, v, t), where u and v are consecutive syscalls and t is the normalized relative timestamp of the transition. Every unique syscall maintains a memory vector $\mathbf{h}_u \in \mathbb{R}^{d_m}$ that is updated as new events arrive. For each

event (u, v, t), a message is computed for both endpoints using their current memories and a time encoding of the elapsed time since their last interaction:

$$\mathbf{m}_u = \text{ReLU}(W\,[\mathbf{h}_u \parallel \mathbf{h}_v \parallel \phi(\Delta t_u)])\,, \quad \mathbf{h}_u \leftarrow \text{GRU}(\mathbf{m}_u,\, \mathbf{h}_u)$$

where $\phi(\Delta t) = \cos(\Delta t \cdot \boldsymbol{\omega} + \boldsymbol{\varphi})$ is a learnable sinusoidal time encoder **?**, and Δt_u is the time elapsed since u last appeared in the trace. The symmetric update is applied to v simultaneously. After processing all events, the graph-level embedding is the mean of all active node memories, which is passed to a two-layer MLP classifier.

Node memories are initialized from a learned per-syscall embedding rather than zeros, giving the model a static prior about each syscall's identity before any events are processed. To keep training tractable, long traces are covered by uniform stride-based subsampling rather than head truncation, ensuring that attack-relevant events occurring at any point in the trace remain represented. Events are processed in temporal chunks of size B_c (default 50), using the memory state from the start of each chunk; this synchronous-update approximation replaces $|E|$ sequential Python dispatches with $|E|/B_c$ batched matrix operations, yielding a substantial speedup at a small cost in temporal fidelity. Class imbalance is addressed by manually weighting each sample's loss by its class weight before gradient accumulation, since `F.cross_entropy` with `weight` cancels itself out at batch size one. Time deltas are normalized to log-microseconds before encoding, $\hat{\Delta}t = \log(1 + 10^6\,\Delta t)$, so that the learnable frequencies $\boldsymbol{\omega}$ produce meaningful phase rotations; raw seconds give $\Delta t \approx 10^{-6}$, collapsing $\cos(\Delta t \cdot \boldsymbol{\omega}) \approx 1$ uniformly across all events. An enhanced variant, **TGN-Hybrid**, additionally seeds each node's initial memory with per-trace static features (transition frequency, in/out degree, PageRank) via a zero-initialized linear projection, so the model starts identically to vanilla TGN and learns to exploit the structural prior incrementally.

Training protocol. All GNN models are trained with Adam and selected using validation F1. The best validation checkpoint is then evaluated on the held-out test split. The report figures and tables are exported automatically from the training pipeline so that the manuscript remains synchronized with the actual experimental artifacts.

Learning paradigms. Beyond standard supervised classification, we evaluate one additional learning setting. In the supervised setting, the encoder and classifier are trained end-to-end with the binary cross-entropy classification objective implemented through the usual softmax cross-entropy loss

$$\mathcal{L}_{\text{sup}} = -\sum_{i=1}^{N} \log p_\theta(y_i \mid G_i).$$

This is the most direct setting when graph labels are available for all training samples.

In the anomaly-detection setting, the encoder is trained mainly on normal graphs. Let g_i denote the graph embedding of a normal sample and let c be the center of the normal embedding cloud. We minimize a one-class objective inspired by Deep SVDD Ruff et al. (2018), where the model learns a compact hypersphere containing normal samples:

$$\mathcal{L}_{\text{ano}} = \frac{1}{N_{\text{normal}}} \sum_{i=1}^{N_{\text{normal}}} \|g_i - c\|_2^2.$$

At inference time, the anomaly score is the squared distance $\|g - c\|_2^2$, and the decision threshold is selected on the validation split. This setting is relevant for intrusion detection because in realistic deployments normal traces are often abundant while attack labels are scarce or incomplete.

Cross-scenario protocol. To study out-of-distribution generalization on LID-DS, we define a transfer setting from `Bruteforce_CWE-307` to `CVE-2014-0160`. We train and tune the model on the source scenario only, then evaluate it unchanged on the target scenario. This protocol deliberately introduces a scenario shift between source and target attacks while keeping model selection entirely on the source scenario.

4.1 Graph Inspection

The generated syscall graphs remain in a small-to-medium regime while staying fairly dense: ADFA-LD contains 5951 graphs with 19.9 nodes and 57.3 edges on average (directed density 0.245);

LID-DS (Bruteforce_CWE-307) contains 907 graphs with 29.5 nodes and 240.5 edges on average (directed density 0.283); LID-DS (CVE-2014-0160) contains 878 graphs with 29.8 nodes and 113.8 edges on average (directed density 0.132). This profile makes local message-passing models appropriate because each node interacts with a limited but informative neighborhood, so a few propagation steps can aggregate most of the useful local transition context without requiring very deep architectures. We therefore compare a plain GCN baseline, GraphSAGE for more robust neighborhood aggregation, GAT to test whether attention over heterogeneous syscall transitions improves discrimination, and our WGCN+ variant that combines weighted GCN propagation, learned syscall embeddings, structural node features, layer normalization, and hybrid graph pooling.

Dataset	Graphs	Attack %	Mean $\|V\|$	Mean $\|E\|$	Mean density	Mean seq. len.
ADFA-LD	5951	12.54	19.94	57.29	0.24	461.70
LID-DS (Bruteforce_CWE-307)	907	15.33	29.47	240.53	0.28	8136.46
LID-DS (CVE-2014-0160)	878	13.67	29.78	113.81	0.13	1640.26

Table 1: Graph-level inspection of the datasets used by the training pipeline, computed on the same trace pools as the reported experiments.

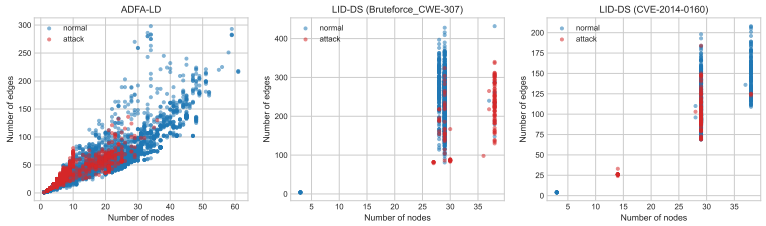

Figure 1: Number of nodes versus number of edges for the generated syscall graphs, colored by class.

5 Evaluation

Experimental setting. The benchmark combines PageRank, GCN, GraphSAGE, GAT, and WGCN+ on ADFA-LD. On LID-DS, we report PageRank and WGCN+ on the two retained scenarios. In addition to this main benchmark, we run a feature ablation on `LID-DS / Bruteforce_CWE-307`, a comparison between supervised and anomaly-detection training for WGCN+ on ADFA-LD, a comparison against sequence-only baselines, and a cross-scenario transfer experiment from `Bruteforce_CWE-307` to `CVE-2014-0160`. The sequence baselines are TF-IDF n-gram logistic regression and a bidirectional GRU. The third scenario initially considered, `CWE-89-SQL-injection`, was removed from the final study because preprocessing and graph construction on this scenario exceeded the available memory budget in our environment.

Main observations. Three conclusions stand out from the benchmark reported in Table 2. First, on ADFA-LD, all GNN variants clearly outperform the PageRank baseline in ROC-AUC and F1, and GAT achieves the best overall test performance. This suggests that, on ADFA-LD, heterogeneous transition importance matters and is captured well by attention. Second, on the LID-DS bruteforce scenario, both approaches are very strong, but the PageRank baseline is actually the best method in our experiments, which indicates that this scenario is dominated by highly regular local transition anomalies that can already be captured by the window-based normal-pattern library. Third, on the harder `CVE-2014-0160` scenario, PageRank remains imperfect but usable, whereas WGCN+ reaches high ROC-AUC yet collapses to zero F1 under the default classification threshold. This means the model still ranks traces reasonably well, but its final decision boundary is poorly calibrated under this scenario.

Feature ablation. The feature ablation in Table 3 confirms that representation design matters substantially on LID-DS. Using only syscall embeddings and binary edges already yields a non-trivial result ($F1 = 0.7843$), which shows that graph topology and syscall identity alone contain useful

information. Adding structural node descriptors improves the result to $F1 = 0.8085$, but the main jump appears when the richer handcrafted features are reintroduced: the full node profile with binary edges reaches $F1 = 0.8679$. Finally, restoring weighted edges gives the best configuration with $F1 = 0.9231$ and $ROC\text{-}AUC = 0.9782$. This progression is consistent with the problem structure: the bruteforce scenario is not characterized only by which transitions occur, but also by how often they are repeated and by the statistics attached to the involved syscalls.

Learning paradigms. The comparison of learning paradigms in Table 4 follows the expected ranking under our current implementation budget. Supervised learning is the strongest setting, with $F1 = 0.8322$ and $ROC\text{-}AUC = 0.9851$, because it directly optimizes the discriminative objective of interest. The anomaly-detection setting is weaker ($F1 = 0.4875$, $ROC\text{-}AUC = 0.8497$), but it remains informative as a realistic deployment scenario in which one primarily models normal behavior and detects deviations by distance in embedding space.

Sequence-only baselines. The sequence baselines in Table 5 produce a nuanced picture. On ADFA-LD, both sequence-only models are competitive: TF-IDF logistic regression reaches $F1 = 0.7692$ and the GRU reaches $F1 = 0.7708$, both with ROC-AUC close to 0.98. This means that on ADFA-LD a large fraction of the signal is already present in local sequential motifs. However, both remain below the best graph models, especially GAT, which suggests that the graph abstraction still captures useful transition structure beyond raw local n-grams. On `LID-DS / Bruteforce_CWE-307`, the TF-IDF baseline is extremely strong ($F1 = 0.9630$, $ROC\text{-}AUC = 1.0000$), on par with the strongest graph-based results, whereas the GRU performs poorly ($F1 = 0.1765$). A plausible explanation is that the bruteforce traces are long and repetitive: a sparse bag of local motifs still captures these repetitions well, while the truncated recurrent encoder struggles with long-range dependencies and with discriminative events that may appear far from the retained sequence prefix.

Cross-scenario generalization. The transfer experiment in Table 6 reveals a clear loss of generalization across attack scenarios. WGCN+ drops to $F1 = 0.3217$ and $ROC\text{-}AUC = 0.7058$ on the target scenario, even though it performs very well on the source validation split. TF-IDF logistic regression degrades further to $F1 = 0.2012$ and ROC-AUC 0.5783, while the GRU almost collapses completely ($F1 = 0.0382$). Interestingly, the PageRank detector remains competitive in this transfer setting ($F1 = 0.3519$, $ROC\text{-}AUC = 0.7045$), likely because its anomaly score is anchored in deviations from normal local transition patterns rather than in a fully discriminative source-task boundary. Overall, these results suggest that the representation learned from one attack family does not transfer cleanly to another on LID-DS, and that scenario shift is a major challenge for host-based intrusion detection.

TGN results on LID-DS. The TGN is evaluated exclusively on LID-DS, since ADFA-LD does not provide per-event timestamps. Vanilla TGN achieves a test ROC-AUC of 0.759 and a test F1 of 0.459, with a decision threshold of 0.509 selected automatically on the validation split. The ROC-AUC confirms that TGN does rank attack traces above normal ones. However, the predicted scores are compressed into a band of width $\sim 10^{-4}$ around 0.5, indicating poor calibration: the model learns a relative ordering but not confident class membership, making the decision threshold fragile. We identify three contributing causes: (i) unweighted cross-entropy on an 85%/15% imbalanced dataset allows the model to minimize loss without separating the classes; (ii) LID-DS inter-syscall deltas are in the microsecond range, so the raw-second time encoding gives $\cos(\Delta t \cdot \boldsymbol{\omega}) \approx 1$ for all events, rendering the temporal signal uninformative; (iii) mean pooling over all active syscall memories dilutes the attack-specific updates of individual nodes. Correcting all three—square-root-balanced class weights, log-microsecond time normalization, and concat(mean, max) graph pooling—yields a model with a well-distributed score range, at a cost in ranking ability (ROC-AUC 0.622, F1 0.311). By contrast, GCN produces a bimodal score distribution with attacks near 1.0 and normals near 0.0 and near-perfect AUC. This gap reveals that static transition structure is not only more discriminative but also better calibrated than per-event temporal dynamics for the attack scenarios studied.

Interpretation and limitations. The graph inspection in Table 1 and Figure 1, the benchmark in Table 2, and the ablations in Table 3 together suggest that there is no single winner across all operating conditions, but that the best method can be understood from the data regime. Dense and

repetitive scenarios such as bruteforce attacks are particularly favorable to local pattern methods: PageRank and TF-IDF both perform extremely well there because they focus on repeated transition motifs. The feature ablation further shows that WGCN+ benefits from preserving both richer node descriptors and transition multiplicities, which is coherent with the fact that these graphs encode repeated process behavior. By contrast, ADFA-LD benefits more from learned graph representations, especially attention-based ones, although the strong ADFA-LD sequence baselines also show that sequential information alone already carries a large portion of the signal. Finally, the transfer experiment highlights a major limitation of all discriminative approaches considered here: training on one attack scenario does not ensure robust generalization to another. The current study nevertheless remains limited by CPU-only training, by the exclusion of the SQL injection scenario, and by the absence of a more systematic threshold-calibration stage for GNN outputs.

5.1 AUTOMATED RESULTS

Dataset	Scenario	Model	Acc.	F1	Prec.	Rec.	ROC-AUC
adfa_ld	all	pagerank	0.8128	0.4652	0.3619	0.6510	0.8092
adfa_ld	all	gnn:wgcn_plus	0.9421	0.7823	0.7381	**0.8322**	0.9706
adfa_ld	all	gnn:gcn	0.9362	0.7164	**0.8067**	0.6443	0.9567
adfa_ld	all	gnn:graphsage	0.9337	0.7208	0.7612	0.6846	0.9617
adfa_ld	all	gnn:gat	**0.9488**	**0.7987**	0.7857	0.8121	**0.9790**
lid_ds	Bruteforce_CWE-307	pagerank	**0.9890**	**0.9630**	**1.0000**	**0.9286**	**1.0000**
lid_ds	Bruteforce_CWE-307	gnn:wgcn_plus	0.9725	0.9020	**1.0000**	0.8214	0.9763
lid_ds	CVE-2014-0160	pagerank	**0.8864**	**0.3750**	**0.7500**	**0.2500**	0.7818
lid_ds	CVE-2014-0160	gnn:wgcn_plus	0.8636	0.0000	0.0000	0.0000	**0.9041**
lid_ds	all	tgn	**0.8156**	**0.4590**	**0.4000**	**0.5385**	**0.7594**

Table 2: Test-set metrics exported automatically from the training pipeline.

5.2 FEATURE ABLATION

Dataset	Scenario	Node features	Edge mode	Acc.	F1	ROC-AUC
lid_ds	Bruteforce_CWE-307	embedding-only	binary	0.9396	0.7843	0.9490
lid_ds	Bruteforce_CWE-307	structural	binary	0.9505	0.8085	0.8929
lid_ds	Bruteforce_CWE-307	full	binary	0.9615	0.8679	0.9675
lid_ds	Bruteforce_CWE-307	full	weighted	**0.9780**	**0.9231**	**0.9782**

Table 3: Ablation of node and edge features for the selected supervised GNN runs.

5.3 LEARNING PARADIGMS

Dataset	Scenario	Paradigm	Model	Acc.	F1	ROC-AUC
adfa_ld	all	supervised	gnn:wgcn_plus	**0.9580**	**0.8322**	**0.9851**
adfa_ld	all	anomaly detection	gnn:wgcn_plus	0.8447	0.4875	0.8497

Table 4: Comparison of supervised and anomaly-detection training settings.

5.4 SEQUENCE BASELINES

Dataset	Scenario	Model	Paradigm	Acc.	F1	ROC-AUC
adfa_ld	all	seq:tfidf-logreg	supervised	0.9270	0.7692	0.9783
adfa_ld	all	seq:gru	supervised	**0.9446**	**0.7708**	**0.9790**
lid_ds	Bruteforce_CWE-307	seq:tfidf-logreg	supervised	**0.9890**	**0.9630**	**1.0000**
lid_ds	Bruteforce_CWE-307	seq:gru	supervised	0.8462	0.1765	0.5831

Table 5: Non-graph baselines trained directly on the syscall sequence.

5.5 CROSS-SCENARIO GENERALIZATION

Train scenario	Test scenario	Model	Paradigm	Src. Acc.	Src. F1	Src. AUC	Tgt. Acc.	Tgt. F1	Tgt. AUC
Bruteforce_CWE-307	CVE-2014-0160	gnn:wgcn_plus	supervised	0.9780	0.9231	0.9692	**0.8895**	0.3217	**0.7058**
Bruteforce_CWE-307	CVE-2014-0160	seq:tfidf-logreg	supervised	0.9890	0.9630	1.0000	0.8462	0.2012	0.5783
Bruteforce_CWE-307	CVE-2014-0160	seq:gru	supervised	0.8571	0.2353	0.5706	0.8280	0.0382	0.5588
Bruteforce_CWE-307	CVE-2014-0160	pagerank	anomaly detection	0.9890	0.9630	1.0000	0.8405	**0.3519**	0.7045

Table 6: Transfer from one LID-DS attack scenario to another. The source columns report validation metrics on the source scenario, while the target columns report test metrics on the target scenario.

5.6 DIAGNOSTICS: ADFA_LD_GAT_DIAGNOSTIC

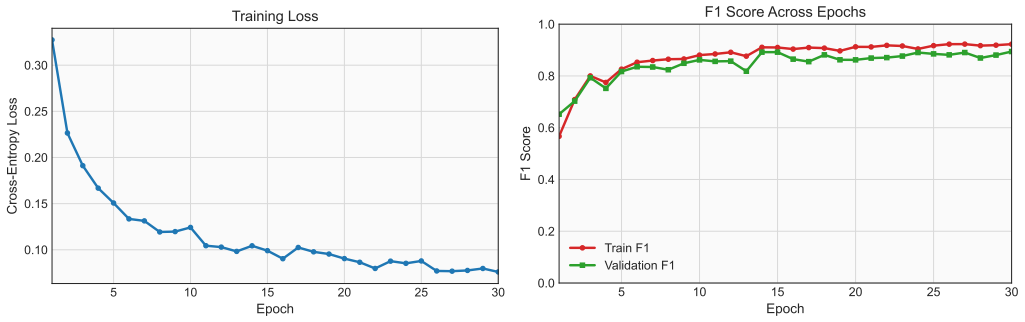

Figure 2: Training diagnostics for `adfa_ld_gat_diagnostic`: cross-entropy loss and F1 score as a function of epoch.

6 CONCLUSIONS

This project shows that syscall graphs form a useful and operational representation for host-based intrusion detection. The graph construction step preserves transition structure while enabling both classical graph scoring and graph neural message passing. In our experiments, GAT provides the strongest results on ADFA-LD, whereas the PageRank baseline remains remarkably effective on the bruteforce LID-DS scenario. The main lesson is therefore not that one model dominates everywhere, but that graph-based modeling exposes structure that different families of methods can exploit in complementary ways.

Several extensions follow naturally from this work. A first direction is to calibrate the GNN decision threshold on the validation split in the same spirit as the PageRank baseline, especially for difficult imbalanced scenarios such as `CVE-2014-0160`. A second direction is to include the SQL injection scenario once a more memory-efficient preprocessing path is available. Finally, we introduced a Temporal Graph Network (TGN) that treats each trace as an ordered stream of timestamped events and updates per-node memory vectors via GRU as events arrive. Vanilla TGN achieves AUC 0.759 on LID-DS but with compressed output scores that make its threshold unreliable; after correcting class weighting, time-encoding scale, and graph pooling, a calibrated variant achieves AUC 0.622 with a proper score distribution. TGN-Hybrid, which seeds initial memories with static per-trace

graph features, is a natural extension in the same range. All TGN variants are significantly outperformed by the static GCN, which reaches near-perfect AUC and F1. This gap is itself a finding: for the attack scenarios studied, the aggregated syscall transition structure is a sufficient and highly discriminative signal, and the temporal ordering of individual events does not provide additional discriminative power beyond what the static graph already captures. This suggests that these attacks change *which* syscalls are used and *how often*, rather than only *when* they occur.

7 REFERENCES

REFERENCES

Gideon Creech and Jiankun Hu. Adfa-ld: A new dataset for host-based intrusion detection. *Proceedings of the 2013 Military Communications and Information Systems Conference*, 2013.

Kiss et al. A modern and sophisticated host-based intrusion detection dataset. *Cybersecurity*, 2021.

Martin Grimmer, Martin Max Röhling, Matthias Kricke, Bogdan Franczyk, and Erhard Rahm. Intrusion detection on system call graphs. In *25. DFN-Konferenz Sicherheit in vernetzten Systemen*, 2018.

William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, 2017.

Steven A. Hofmeyr, Stephanie Forrest, and Anil Somayaji. Intrusion detection using sequences of system calls. *Journal of Computer Security*, 6(3):151–180, 1998. doi: 10.3233/JCS-980109.

Thomas Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.

Quan Qian, Jianyu Li, Jing Cai, Rui Zhang, and Mingjun Xin. An anomaly intrusion detection method based on pagerank algorithm. In *2013 IEEE International Conference on Green Computing and Communications and IEEE Internet of Things and IEEE Cyber, Physical and Social Computing*, pp. 2226–2230. IEEE, 2013.

Lukas Ruff, Robert A. Vandermeulen, Nico Görnitz, Lucas Deecke, Shoaib Ahmed Siddiqui, Alexander Binder, Emmanuel Müller, and Marius Kloft. Deep one-class classification. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 4393–4402, 2018.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.

Christina Warrender, Stephanie Forrest, and Barak Pearlmutter. Detecting intrusions using system calls: Alternative data models. In *1999 IEEE Symposium on Security and Privacy*, pp. 133–145. IEEE, 1999.